

1. 策略梯度定理

1.1 目标函数

策略优化的目标是最大化期望累积回报

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

其中轨迹 $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ 。

1.2 策略梯度定理

定理 (Sutton et al., 1999)

策略梯度可以表示为：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right]$$

其中 $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ 是从时刻 t 开始的回报。

1.3 推导过程

关键步骤：

$$\nabla_\theta J(\theta) = \nabla_\theta \int_{\tau} P(\tau|\theta) R(\tau) d\tau$$

使用对数导数技巧 (log-derivative trick)：

$$\nabla_\theta P(\tau|\theta) = P(\tau|\theta) \nabla_\theta \log P(\tau|\theta)$$

由于轨迹概率：

$$P(\tau|\theta) = p(s_0) \prod_{t=0}^T \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

取对数并求梯度，**环境动力学项消失**（与 θ 无关）：

$$\nabla_{\theta} \log P(\tau|\theta) = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

2. REINFORCE 算法

2.1 算法思想

REINFORCE 是最简单的策略梯度算法，直接使用蒙特卡洛采样估计梯度：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) G_t^{(i)}$$

2.2 算法流程

```
1  算法：REINFORCE
2  输入：策略网络  $\pi_{\theta}$ ，学习率  $\alpha$ 
3  输出：优化后的策略  $\pi_{\theta}$ 
4
5  1. for episode = 1 to M:
6    2.   收集轨迹  $\tau = (s_0, a_0, r_0, \dots, s_T, a_T, r_T)$ 
7    3.   for t = 0 to T:
8      4.     计算回报:  $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ 
9      5.     计算梯度:  $g_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$ 
10   6.   更新参数:  $\theta \leftarrow \theta + \alpha \cdot \sum_t g_t$ 
11  7. return  $\pi_{\theta}$ 
```

2.3 高方差问题

REINFORCE 的核心问题是高方差

$$\text{Var}[\hat{g}] \propto \text{Var}[G_t]$$

由于蒙特卡洛采样，回报 G_t 的方差很大，导致：

训练不稳定

收敛缓慢

需要大量样本

3. 方差降低技术

3.1 基线 (Baseline)

关键洞察：从回报中减去不依赖于动作的基线 $b(s)$ ，不改变梯度期望，但可以显著降低方差。

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a|s)(G_t - b(s))]$$

最优基线：

$$b^*(s) = \frac{\mathbb{E}[(\nabla_{\theta} \log \pi)^2 G_t]}{\mathbb{E}[(\nabla_{\theta} \log \pi)^2]}$$

实践中：使用状态值函数 $V(s)$ 作为基线。

3.2 优势函数 (Advantage Function)

定义优势函数：

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

直观含义：动作 a 相对于平均水平好多少。

使用优势的策略梯度：

$$\nabla_{\theta} J = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a|s) A^{\pi}(s, a)]$$

3.3 广义优势估计 (GAE)

GAE (Generalized Advantage Estimation) 在偏差和方差之间取得平衡

$$\hat{A}_t^{GAE(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

其中TD误差 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 。

实际计算：

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

4. TRPO：信任域策略优化

4.1 动机

策略梯度更新可能导致策略**崩溃**：一次过大的更新使策略变得很差。

TRPO (Trust Region Policy Optimization) 的思想：限制每次更新的幅度。

4.2 优化问题

TRPO 将策略优化形式化为约束优化问题：

$$\max_{\theta} \mathbb{E}_{s \sim \rho_{\theta_{old}}, a \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} A^{\theta_{old}}(s, a) \right]$$

$$\text{s.t.} \quad \mathbb{E}_{s \sim \rho_{\theta_{old}}} [D_{KL}(\pi_{\theta_{old}}(\cdot|s) \parallel \pi_{\theta}(\cdot|s))] \leq \delta$$

4.3 重要性采样比率

定义重要性采样比率：

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

目标函数可以写为：

$$L(\theta) = \mathbb{E}_t [r_t(\theta)A_t]$$

5. PPO：近端策略优化

5.1 核心思想

PPO (Proximal Policy Optimization) 是 TRPO 的简化版本，用**裁剪**代替硬约束。

5.2 PPO-Clip 目标函数

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

其中 ϵ 通常取 0.1~0.2。

直观理解：

- 当 $A_t > 0$ (好动作) : r_t 增大有利，但被限制在 $1 + \epsilon$
- 当 $A_t < 0$ (坏动作) : r_t 减小有利，但被限制在 $1 - \epsilon$

5.3 完整目标函数

PPO 通常使用复合损失：

$$L(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta]$$

其中：

- L^{VF} ：值函数损失
- $S[\pi_\theta]$ ：熵奖励（鼓励探索）

5.4 PPO 算法流程

```

1  算法: PPO (Proximal Policy Optimization)
2  输入: 策略网络  $\pi_\theta$ , 值网络  $V_\phi$ , 超参数
3  输出: 优化后的策略
4
5  1. for iteration = 1 to N:
6      2. 使用  $\pi_\theta$  收集  $T$  步轨迹数据
7      3. 计算优势估计  $\hat{A}_t$  (使用GAE)
8      4. for epoch = 1 to K:
9          5. for mini-batch in 数据:
10             6. 计算比率  $r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\{\theta_{old}\}}(a_t|s_t)$ 
11             7. 计算裁剪目标:
12                  $L^{CLIP} = E[\min(r_t \cdot \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t)]$ 
13             8. 计算值函数损失:
14                  $L^{VF} = (V_\phi(s_t) - R_t)^2$ 
15             9. 计算熵奖励:
16                  $S = -E[\log \pi_\theta(a|s)]$ 
17             10. 更新  $\theta$  和  $\phi$ 
18             11.  $\theta_{old} \leftarrow \theta$ 
19             12. return  $\pi_\theta$ 

```

版权声明：本文为 CSDN 博主「Robot 侠」的原创文章，遵循 CC 4.0 BY-SA 版权协议，转载请附上原文出处链接及本声明。

原文链接：https://blog.csdn.net/weixin_52694742/article/details/156738179